

# Assignment 20.3

---

Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code

Name: Manupati Eshwar

Hall Ticket No: 2303A51495

## Task 1 – Password Strength Validation

### Prompt

Generate a Python user registration program

### AI Generated Code (Insecure)

```
username = input("Enter username: ")  
  
password = input("Enter password: ")
```

```
if len(password) > 0:  
    print("User registered successfully!")  
else:  
    print("Invalid password")
```

### Issues Identified

- No minimum password length
- Accepts weak passwords like "123"
- No complexity check
- No protection against common passwords

### Secure Code (Fix)

```
import re
```

```
def is_strong_password(password):
```

```

if len(password) < 8:
    return False

if not re.search("[A-Z]", password):
    return False

if not re.search("[a-z]", password):
    return False

if not re.search("[0-9]", password):
    return False

if not re.search("[@#$%^&+=]", password):
    return False

return True

username = input("Enter username: ")
password = input("Enter password: ")

common_passwords = ["123456", "password", "admin"]

if password in common_passwords:
    print("Weak password! Choose another.")
elif is_strong_password(password):
    print("User registered successfully!")
else:
    print("Password must be strong!")

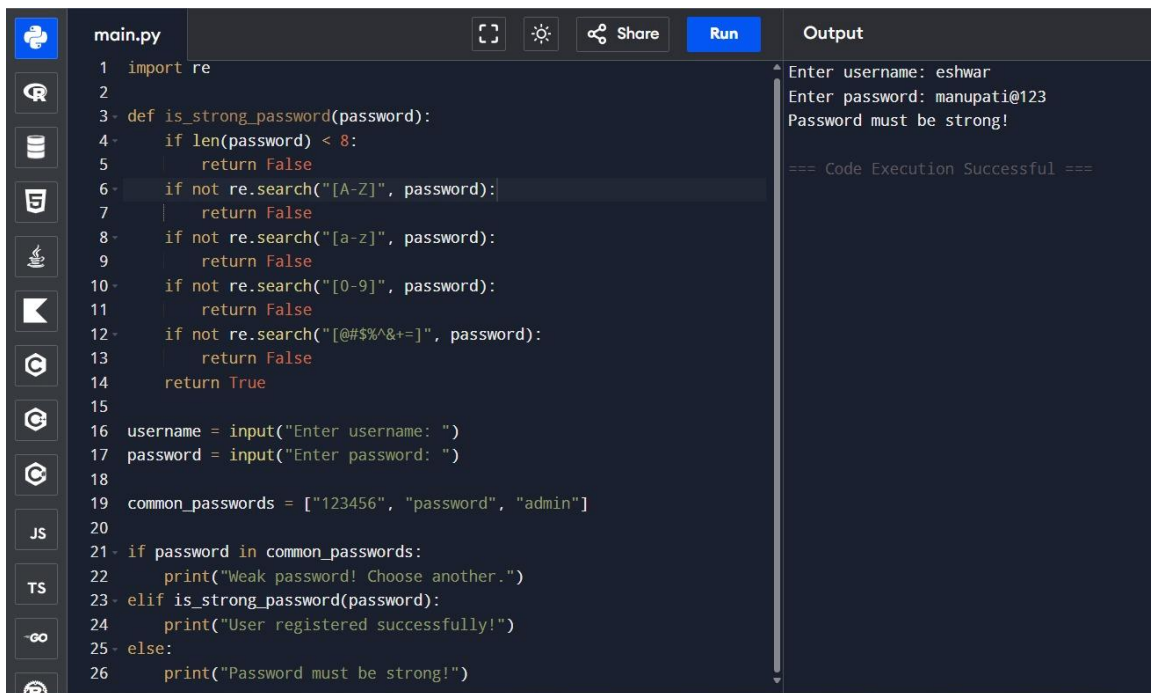
```

### **Explanation**

Adds validation rules

### **Output**

Strong accepted, weak rejected



```
main.py
1 import re
2
3 def is_strong_password(password):
4     if len(password) < 8:
5         return False
6     if not re.search("[A-Z]", password):
7         return False
8     if not re.search("[a-z]", password):
9         return False
10    if not re.search("[0-9]", password):
11        return False
12    if not re.search("[@#$$%^&+=]", password):
13        return False
14    return True
15
16 username = input("Enter username: ")
17 password = input("Enter password: ")
18
19 common_passwords = ["123456", "password", "admin"]
20
21 if password in common_passwords:
22     print("Weak password! Choose another.")
23 elif is_strong_password(password):
24     print("User registered successfully!")
25 else:
26     print("Password must be strong!")
```

Output

```
Enter username: eshwar
Enter password: manupati@123
Password must be strong!

=== Code Execution Successful ===
```

## Task 2 – File Upload Security

### Prompt

Generate Flask file upload app

### AI Generated Code (Insecure)

from flask import Flask, request

```
app = Flask(__name__)
```

```
@app.route('/upload', methods=['POST'])
```

```
def upload():
```

```
    file = request.files['file']
```

```
    file.save(file.filename)
```

```
    return "File uploaded"
```

```
app.run()
```

## Issues Identified

No validation or size limit

## Secure Code (Fix)

```
from flask import Flask, request
```

```
import os
```

```
app = Flask(__name__)
```

```
UPLOAD_FOLDER = 'uploads'
```

```
ALLOWED_EXTENSIONS = {'png', 'jpg', 'pdf'}
```

```
MAX_FILE_SIZE = 2 * 1024 * 1024 # 2MB
```

```
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
```

```
def allowed_file(filename):
```

```
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
```

```
@app.route('/upload', methods=['POST'])
```

```
def upload():
```

```
    file = request.files['file']
```

```
    if not allowed_file(file.filename):
```

```
        return "Invalid file type"
```

```
    if len(file.read()) > MAX_FILE_SIZE:
```

```
        return "File too large"
```

```
file.seek(0)
```

```
filepath = os.path.join(app.config['UPLOAD_FOLDER'], file.filename)
```

```
file.save(filepath)
```

```
return "Secure upload successful"
```

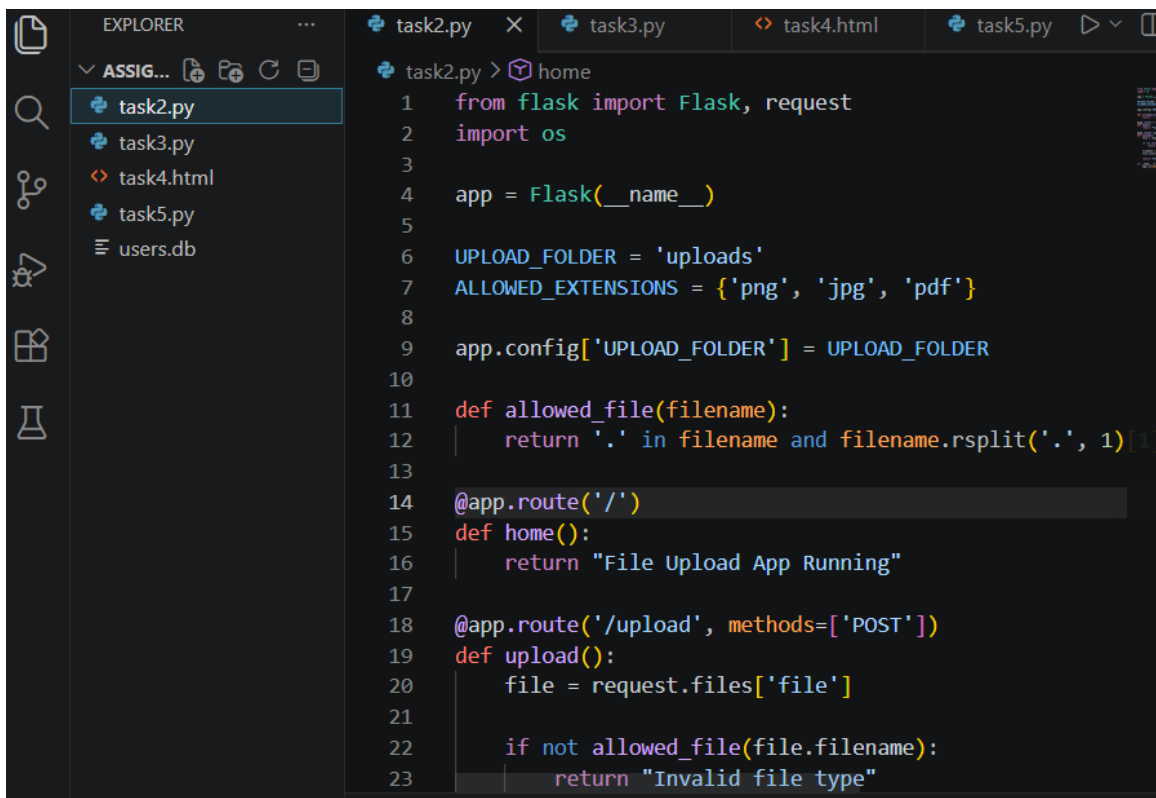
```
app.run()
```

## Explanation

- Validates file types
- Restricts file size
- Uses safe storage

## Output

Safe upload only



The screenshot shows a code editor with a dark theme. On the left is the 'EXPLORER' sidebar showing a file tree with 'task2.py', 'task3.py', 'task4.html', 'task5.py', and 'users.db'. The main editor area displays the code for 'task2.py'. The code is as follows:

```
task2.py > home
1  from flask import Flask, request
2  import os
3
4  app = Flask(__name__)
5
6  UPLOAD_FOLDER = 'uploads'
7  ALLOWED_EXTENSIONS = {'png', 'jpg', 'pdf'}
8
9  app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
10
11 def allowed_file(filename):
12     return '.' in filename and filename.rsplit('.', 1)[1]
13
14 @app.route('/')
15 def home():
16     return "File Upload App Running"
17
18 @app.route('/upload', methods=['POST'])
19 def upload():
20     file = request.files['file']
21
22     if not allowed_file(file.filename):
23         return "Invalid file type"
```

File Upload App Running

### Task 3 – API Security

#### Prompt

Generate API endpoint

#### AI Generated Code (Insecure)

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/data')
```

```
def get_data():
```

```
    return {"message": "Public data"}
```

```
app.run()
```

#### Issues Identified

- No authentication
- Public access
- No security control

### Secure Code (Fix)

```
from flask import Flask, request, jsonify

app = Flask(__name__)

API_KEY = "secure123"

def authenticate(req):

    return req.headers.get("API-Key") == API_KEY

@app.route('/data', methods=['GET'])
def get_data():

    if not authenticate(request):

        return jsonify({"error": "Unauthorized"}), 401

    return jsonify({"message": "Secure data"})

app.run()
```

### Explanation

- 🔖 Adds API key authentication
- 🔖 Restricts unauthorized users

### Output

Secure response

```
4
5  API_KEY = "12345"
6
7  @app.route('/')
8  def home():
9      return "API Server Running"
10
11 @app.route('/data', methods=['GET'])
12 def get_data():
13     key = request.headers.get("API-Key")
14
15     if key == API_KEY:
16         return jsonify({"message": "Authorized Access"})
17     else:
18         return jsonify({"error": "Unauthorized"}), 401
19
20 if __name__ == '__main__':
21     app.run(debug=True)
```

API Server Running

Pretty-print ☐

```
{
  "error": "Unauthorized"
}
```



## Task 4 – XSS Prevention

### Prompt

Generate feedback form

### AI Generated Code (Insecure)

```
<input type="text" id="name">

<button onclick="display()">Submit</button>


<p id="output"></p>


<script>

function display() {

    var name = document.getElementById("name").value;

    document.getElementById("output").innerHTML = name;

}

</script>
```

### Issues Identified

XSS vulnerability

- ❑ Uses innerHTML
- ❑ Allows XSS attacks

### Secure Code (Fix)

```
textContent=userInput

<input type="text" id="name">

<button onclick="display()">Submit</button>


<p id="output"></p>


<script>
```

```
function display() {  
    var name = document.getElementById("name").value;  
    document.getElementById("output").textContent = name;  
}  
</script>
```

### Explanation

Prevents script execution

### Output

Safe display

## Feedback Form

hello eshwar

---

## Feedback Form

<script>alert("hack")</script>

## Task 5 – SQL Injection

### Prompt

Generate DB script

### AI Generated Code (Insecure)

```
import sqlite3
```

```
conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Enter username: ")

query = "SELECT * FROM users WHERE username = '" + username + "'"
cursor.execute(query)

print(cursor.fetchall())
```

### **Issues Identified**

SQL Injection risk

### **Secure Code (Fix)**

```
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

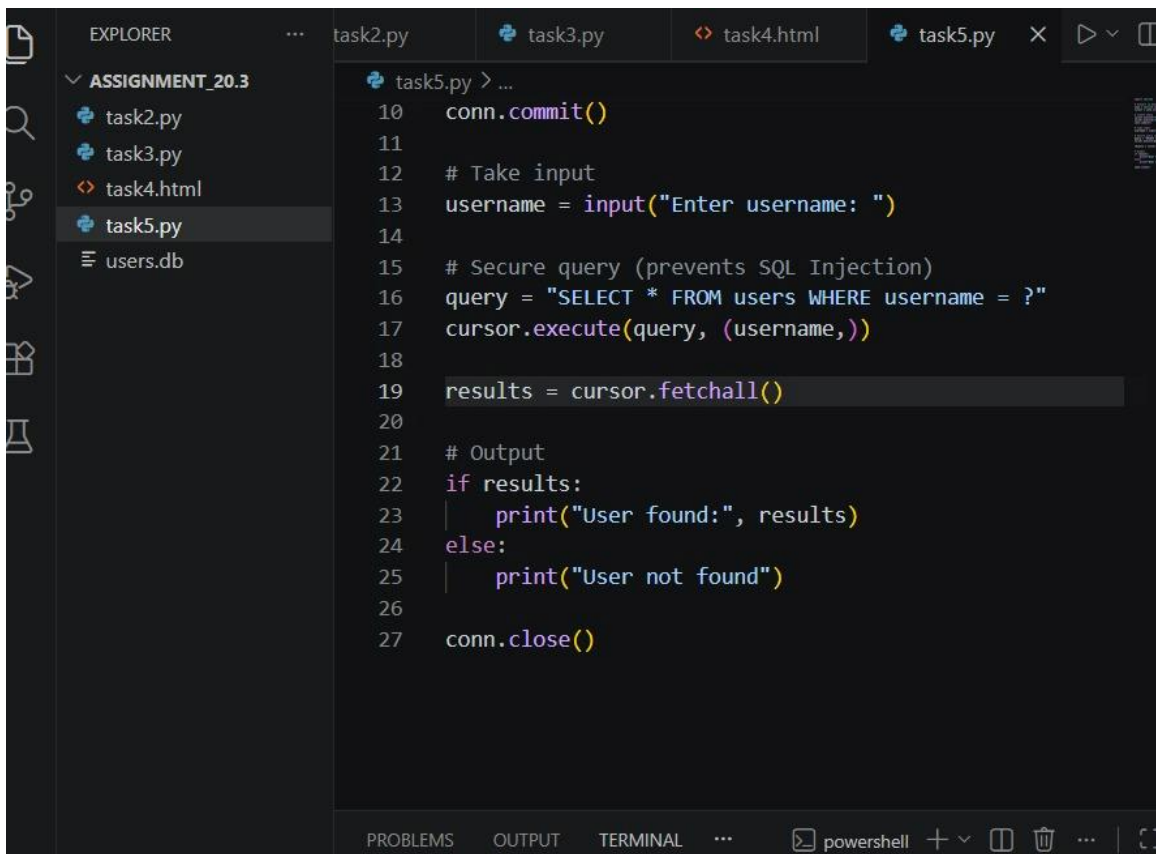
username = input("Enter username: ")

query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,))

print(cursor.fetchall())
```

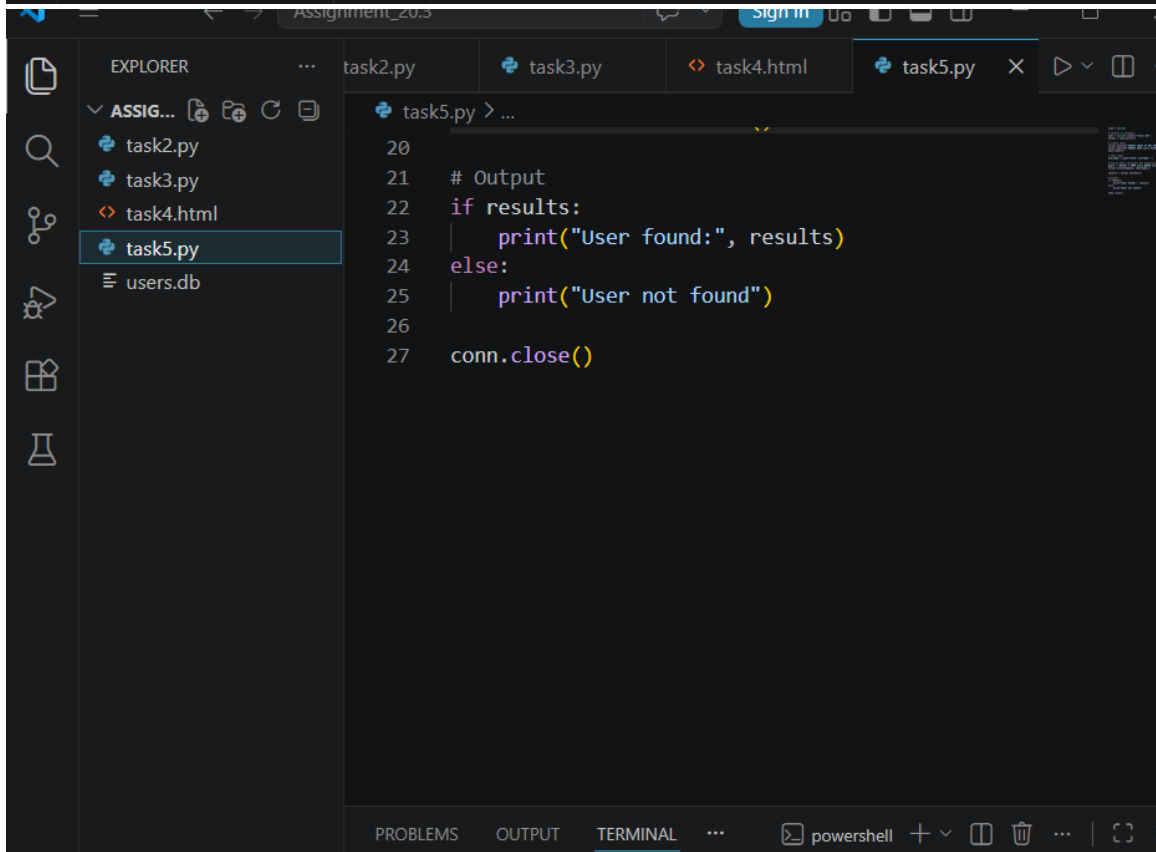
### **Explanation**

Prevents injection



task5.py > ...

```
10 conn.commit()
11
12 # Take input
13 username = input("Enter username: ")
14
15 # Secure query (prevents SQL Injection)
16 query = "SELECT * FROM users WHERE username = ?"
17 cursor.execute(query, (username,))
18
19 results = cursor.fetchall()
20
21 # Output
22 if results:
23     print("User found:", results)
24 else:
25     print("User not found")
26
27 conn.close()
```



task5.py > ...

```
20
21 # Output
22 if results:
23     print("User found:", results)
24 else:
25     print("User not found")
26
27 conn.close()
```

## **Conclusion**

AI-generated code may contain security vulnerabilities. By applying proper validation, authentication, and secure coding practices, we can improve application security, reliability, and performance